

Caso 3 — Red municipal de monitoreo de calidad del aire

Prototype

Factory Method

Builder

Duración: 2 horas

Lenguaje: Java (SE 17+)

Modalidad: individual, aplicación de consola

1. Contexto

La **Secretaría de Ambiente** de una ciudad intermedia opera una red de puestos de monitoreo de calidad del aire. No todos los puestos son iguales: hay estaciones fijas de referencia (equipos costosos, alta precisión), unidades móviles montadas en un vehículo que se desplazan a zonas con quejas ciudadanas, y una malla de sensores de bajo costo instalados en colegios, que son baratos pero cuyas lecturas deben corregirse porque la humedad relativa las infla.

Todos los días a las 7:00 a.m. la Secretaría debe publicar un **boletín de calidad del aire** con el índice de cada estación. Actualmente ese boletín se arma a mano en un procesador de texto y ya se publicaron dos boletines con datos de sensores de bajo costo *sin corregir*, reportando una alerta roja inexistente que generó un lío mediático. Además, configurar una estación nueva toma dos días porque hay que escribir de cero sus contaminantes, umbrales y plan de mantenimiento, cuando en realidad el 90 % es idéntico a otra estación ya existente.

2. Reglas de negocio

2.1 Tipos de puesto y sus analizadores

Cada tipo de puesto instala un analizador distinto para el mismo contaminante. El analizador define la corrección que se aplica a la lectura cruda:

Tipo de puesto	Analizador que instala	Corrección sobre la lectura cruda	Incertidumbre
FIJO_REFERENCIA	Analizador de método de referencia	lectura × factorCalibracion (dato del equipo, p. ej. 1.02)	± 2 %
MOVIL	Analizador óptico portátil	lectura × 1.05 + 0.8 (deriva por transporte)	± 8 %
BAJO_COSTO	Sensor de bajo costo con corrección por humedad	si HR > 75 %: lectura / (1 + 0.012 × (HR – 75)); si no: lectura	± 20 %

HR = humedad relativa en porcentaje, registrada junto con cada lectura.

2.2 Validez del promedio diario

Un contaminante tiene promedio diario válido solo si se cuenta con **al menos 18 de las 24 lecturas horarias**. Con menos, se reporta **DATO INSUFICIENTE** y ese contaminante no participa en el índice. Las lecturas negativas o superiores al rango del equipo se descartan como inválidas antes de contar.

2.3 Cálculo del índice de calidad del aire (ICA)

Para cada contaminante se interpola linealmente dentro del tramo que contiene la concentración corregida:

$$ICA = ((I_{hi} - I_{lo}) / (C_{hi} - C_{lo})) \times (C - C_{lo}) + I_{lo}$$

Tramo ICA	PM2.5 (µg/m³, 24 h)	O3 (ppb, 8 h)	NO2 (ppb, 1 h)	Categoría
0 – 50	0.0 – 12.0	0 – 54	0 – 53	Buena
51 – 100	12.1 – 37.4	55 – 70	54 – 100	Aceptable
101 – 150	37.5 – 55.4	71 – 85	101 – 360	Dañina a grupos sensibles
151 – 200	55.5 – 150.4	86 – 105	361 – 649	Dañina a la salud
201 – 300	150.5 – 250.4	106 – 200	650 – 1249	Muy dañina

El **ICA de la estación** es el **máximo** de los ICA de sus contaminantes válidos, y el contaminante que lo produce se denomina *contaminante crítico*. El ICA se reporta como entero redondeado.

2.4 Configuraciones reutilizables

La Secretaría mantiene **configuraciones modelo** por tipo de zona (residencial, tráfico intenso, industrial) con: contaminantes a medir, intervalo de muestreo, mapa de umbrales de alerta por contaminante y plan de mantenimiento (lista de tareas con periodicidad). Al desplegar una estación nueva se toma la configuración modelo, se duplica y se ajusta. **Una estación no puede quedar compartiendo el mapa de umbrales ni la lista de mantenimientos con la configuración modelo.**

3. Requerimiento

Construya en Java una aplicación de consola que procese la jornada de 24 horas de varias estaciones, calcule el ICA de cada una y emita el boletín diario.

4. Patrones obligatorios (exactamente tres)

Factory Method — creación del analizador según el tipo de puesto

Defina la clase abstracta `PuestoDeMonitoreo` con un método público `procesarJornada(List<LecturaCruda>)` que orquesta el proceso (descartar inválidas, corregir, verificar validez, promediar) y un **método fábrica** `protected abstract Analizador crearAnalizador(String contaminante)`. Implemente `PuestoFijoReferencia`, `PuestoMovil` y `PuestoBajoCosto`, cada uno devolviendo su analizador con la corrección de la tabla 2.1. La corrección debe vivir en el analizador, no en condicionales dentro del puesto.

Prototype — configuraciones modelo de estación

`ConfiguracionEstacion` debe implementar clonación **profunda** del mapa de umbrales (`Map<String,Double>`), de la lista de contaminantes y de la lista de objetos `TareaMantenimiento`. Demuestre en la ejecución que cambiar un umbral y agregar una tarea en una estación clonada no afecta a la configuración modelo ni a otra estación clonada de la misma.

Builder — armado del boletín diario

`BoletinDiario` debe ser inmutable y construirse con un `Builder` fluido.

Obligatorios: fecha, entidad emisora, lista de resultados por estación, ICA máximo de la ciudad y categoría de la ciudad.

Opcionales: recomendaciones a población sensible, pronóstico para el día siguiente, anexo de datos faltantes, responsable de la publicación, nota metodológica.

`build()` debe lanzar `IllegalStateException` si falta un obligatorio, si la lista de estaciones está vacía, o si la categoría es "Dañina a la salud" o peor y no se incluyeron recomendaciones a población sensible.

5. Componente algorítmico obligatorio

1. **Depuración:** descartar lecturas negativas o fuera del rango del analizador, contando cuántas se descartaron.
2. **Corrección:** aplicar la fórmula del analizador correspondiente a cada lectura cruda.
3. **Validez y promedio:** aplicar la regla 2.2 y calcular el promedio de 24 h con dos decimales.
4. **ICA por contaminante:** implementar la interpolación lineal por tramos de la tabla 2.3 mediante una estructura de datos de puntos de corte, **no** con una cadena de `if` escritos a mano por contaminante.
5. **ICA de estación:** máximo entre contaminantes válidos, con identificación del contaminante crítico y su categoría.
6. **Ranking:** ordenar las estaciones por ICA descendente y, a igualdad, por nombre alfabético.
7. **Cobertura de la red:** porcentaje de datos válidos sobre el total esperado (estaciones × contaminantes × 24).

6. Programa de demostración

1. Cree la configuración modelo "Zona residencial" y clónela para dos estaciones; cambie un umbral y agregue una tarea de mantenimiento en una de ellas. Imprima las tres para evidenciar la independencia.
2. Procese la jornada de **cuatro estaciones**: dos fijas de referencia, una móvil y una de bajo costo.
3. La estación de bajo costo debe tener lecturas con humedad superior a 75 % para que se note la corrección: imprima lectura cruda y corregida de al menos tres horas.
4. Una de las estaciones debe quedar con menos de 18 datos válidos en un contaminante para evidenciar `DATO INSUFICIENTE`.
5. Arme y publique el boletín. Intente además construir un boletín en categoría "Dañina a la salud" sin recomendaciones, dentro de un `try/catch`, y muestre el mensaje de la excepción.

Formato sugerido de salida

```
=== SECRETARIA DE AMBIENTE - RED DE CALIDAD DEL AIRE ===
Config modelo 'Zona residencial' umbral PM2.5=37.5 tareas: 3
  Estacion Parque Centenario      umbral PM2.5=37.5 tareas: 3
  Estacion Colegio San Jose       umbral PM2.5=25.0 tareas: 4  <- modificada
  Verificacion del modelo -> umbral PM2.5=37.5 tareas: 3 (intacto)

--- PROCESAMIENTO POR ESTACION ---
[Colegio San Jose] tipo BAJO_COSTO - sensor con correccion por humedad
  h07 cruda 44.20 HR 88 % -> corregida 38.03
  h08 cruda 51.60 HR 91 % -> corregida 43.08
  lecturas descartadas: 2 | validas: 21 de 24 -> VALIDO
  PM2.5 promedio 24h: 39.14 ug/m3 -> ICA 106 (Danina a grupos sensibles)
  03 validas: 15 de 24 -> DATO INSUFICIENTE

--- BOLETIN DIARIO 2026-08-14 ---
# ESTACION      ICA  CATEGORIA      CRITICO
1 Colegio San Jose      106  Danina a grupos sensibles  PM2.5
2 Av. Quebradaseca      88   Aceptable           N02
3 Parque Centenario     46   Buena                PM2.5
4 Unidad movil - Zona Ind. 41   Buena                03
ICA de la ciudad: 106 (Danina a grupos sensibles)
Cobertura de datos validos de la red: 87.5 %
Recomendaciones: personas con asma y ninos deben evitar actividad fisica al aire libre.
```

7. Restricciones y entregables

- Java estándar, sin frameworks ni librerías externas. Las lecturas van quemadas en código o generadas con `Random` con semilla fija para que el resultado sea reproducible.
- Paquetes sugeridos: `puesto`, `analizador`, `configuracion`, `ica`, `boletin`, `app`.
- Entregue el proyecto comprimido (.zip) con código compilable y captura de la salida.
- Incluya `PATRONES.md` (máximo una página) con la ubicación y justificación de cada patrón.

Fuera de alcance

Interfaz gráfica, mapas, lectura de sensores reales o de archivos, persistencia, pruebas automatizadas y envío de correos o notificaciones.